```
void bestfit(int blocks[], int m, int processes[], int n){
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++){
        int bestIdx = -1;
        for(int j=0; j<m; j++){
            if(blocks[j] >= processes[i]){
                if(bestIdx == -1 || blocks[j] < blocks[bestIdx])
                    bestIdx = j;
            }
        }
        if(bestIdx != -1){
            allocation[i] = bestIdx;
            blocks[bestIdx] -= processes[i];
        }
    }
    printf("Best fit Allocation:\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for(int i=0; i<n; i++){
        printf("%d\t\t%d\t\t", i+1, processes[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}
void worstfit(int blocks[], int m, int processes[], int n){
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++){
        int worstIdx = -1;
        for(int j=0; j<m; j++){
            if(blocks[j] >= processes[i]){
                if(worstIdx == -1 || blocks[j] > blocks[worstIdx])
                    worstIdx = j;
            }
        }
        if(worstIdx != -1){
            allocation[i] = worstIdx;
            blocks[worstIdx] -= processes[i];
        }
    }
    printf("\nWorst Fit Allocation:\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for(int i=0; i<n; i++){
        printf("%d\t\t%d\t\t", i+1, processes[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}
int main(){
    int blocks[MAX], processes[MAX];
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    printf("Enter sizes of blocks:\n");
    for(int i=0; i<m; i++)
        scanf("%d", &blocks[i]);
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter sizes of processes:\n");
    for(int i=0; i<n; i++)
```

```
    scanf("%d", &processes[i]);
int blocks1[MAX], blocks2[MAX], blocks3[MAX];
for (int i=0; i<m; i++) {
    blocks1[i] = blocks[i];
    blocks2[i] = blocks[i];
    blocks3[i] = blocks[i];
}
firstfit(blocks1, m, processes, n);
bestfit(blocks2, m, processes, n);
worstfit(blocks3, m, processes, n);
return 0;
}
```

Output:

```
Enter no of memory blocks: 5
Enter size of memory blocks:
400 700 200 300 600
Enter no of processes: 4
Enter size of processes: 212 512 312 526
First fit Allocation
1    212    1
2    512    2
3    312    5
4    526    Not allocated
Best fit Allocation
1    212    4
2    512    5
3    312    1
4    526    2
Worst fit Allocation
1    212    2
2    512    5
3    312    1
4    526    Not allocated
```

Write a C programs to simulate page replacement algorithms.
a) FIFO b) LRU c) Optimal.

```
#include <stdio.h>
int findLRU(int time[], int n) {
    int i, min = time[0], pos = 0;
    for (i=1; i<n; i++) {
        if (time[i] < min) {
            min = time[i];
            pos = i;
        }
    }
    return pos;
}
int main() {
    int pages[30], frames[10], time[10];
    int n, f, i, j, k, faults, counter, pos, found;
    printf("Enter number of pages:");
    scanf("%d", &n);
    printf("Enter page reference string.");
    for(i=0; i<n; i++) {
        scanf("%d", &pages[i]);
    printf("Enter no of frames");
    scanf("%d", &f);
    for(i=0; i<f; i++)
        frames[i] = -1;
    faults = 0; pos = 0;
    printf("In FIFO page replacement \n");
    for(i=0; i<n; i++) {
        found = 0;
        for(j=0; i<f; j++) {
            if(frames[j] == pages[i])
```

```
        found = 1;
    if (found){
        frames[pos] = pages[i];
        pos = (pos+1) % f;
        faults++; }
    printf("Page %d → ", pages[i]);
    for (k=0; k<f; k++)
        (frames[k] != -1) ? printf("%d ",
            frames[k]) : printf("- ");
    printf("\n"); }
printf("Total page faults = %d\n", faults);

for (i=0; i<f; i++) frames[i] = -1;
faults=0; counter=0;
printf("\n LRU Page Replacement:\n");
for (i=0; i<n; i++)
    found=0;
    for (j=0; j<f; j++){
        if (frames[j] == pages[i];
    }
    found = 1;
    counter++;
    time[j] = counter;
    break;
} if (!found){
    if (i<f)
        frames[i] = pages[i]
    else {
        pos = findLRU(time, f);
        frames[pos] = pages[i];
} counter++;
time[i%f] = counter;
faults++; }
```

```
printf("Page %d → ", pages[i]);
for (k=0; k<f; k++)
    (frames[k] != -1) ? printf("%d ",
        frames[k]) : printf("-1");
printf("\n");
}

for (i=0; i<f; i++) frames[i] = -1;
faults = 0;
printf("\n Optimal page Replacement:");
for (i=0; i<n; i++) {
    found=0;
    for(j=0; j<f; j++)
        if (frames[j]==pages[i] found=1;
if (!found) {
    if (i<j)
        frames[i] = pages[i];
    else {
        int future[10], flag[10];
        for(j=0; j<f; j++){
            flag[j]=0;
        for(k=i+1; k<n; k++) {
            if (frames[j]==pages[k]){
                future[j]=k;
                flag[j]=1;
                break;}
        }
    if(!flag[j]) future[j]=n+1;
}
int max = future[0];
pos=0;
for(j=1; j<f; j++){
```

```
            if (future[j] > max){
                max = future[j];
                pos = j;
            }
        }
        frames[pos] = pages[i]; }
        faults++; }
        printf("Total page faults = %d", faults);
        return 0;
}
```

Output:

```
Enter number of pages: 20
Enter page reference string: 7 0 1 2 0 3 0
4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 4
```

FIFO Page Replacement:

~~Page #~~

```
7 → 7 - - -   f
0 → 7 0 - -   f
1 → 7 0 1 -   f
2 → 7 0 1 2   f
0 → 7 0 1 2   h
3 → 3 0 1 2   f
0 → 3 0 1 2   h
4 → 3 4 1 2   f
2 → 3 4 1 2   h
3 → 3 4 1 2   h
0 → 3 4 0 2   f
3 → 3 4 0 2   h
2 → 3 4 0 2   h
1 → 3 4 0 1   f
2 → 2 4 0 1   f
0 → 2 4 0 1   h
1 → 2 4 0 1   h
7 → 2 7 0 1   f
0 → 2 7 0 1   h
1 → 2 7 0 1   h
```

Total page faults (FIFO) = 10

LRU Page Replacement:

```
7 → 7 - - -   f
0 → 7 0 - -   f
1 → 7 0 1 -   f
2 → 7 0 1 2   f
0 → 7 0 1 2   h
3 → 3 0 1 2   f
0 → 3 0 1 2   h
4 → 4 0 1 2   f
2 → 4 0 1 2   h
3 → 3 0 1 2   f
0 → 3 0 1 2   h
3 → 3 0 1 2   h
2 → 3 0 1 2   h
1 → 3 0 1 2   h
2 → 3 0 1 2   h
0 → 3 0 1 2   h
1 → 3 0 1 2   h
7 → 7 0 1 2   f
0 → 7 0 1 2   h
1 → 7 0 1 2   h
```

Total page faults (LRU) = 8

Date __/__/__
Page _____

Optimal Page Replacement:

7 →	7 _ _ _	f
0 →	7 0 _ _	f
1 →	7 0 1 _	f
2 →	7 0 1 2	f
0 →	7 0 1 2	h
3 →	3 0 1 2	f
0 →	3 0 1 2	h
4 →	3 0 4 2	f
2 →	3 0 4 2	h
3 →	3 0 4 2	h
0 →	3 0 4 2	h
3 →	3 0 4 2	h
2 →	3 0 4 2	h
1 →	1 0 4 2	f
2 →	1 0 4 2	h
0 →	1 0 4 2	h
1 →	1 0 4 2	h
7 →	1 0 7 2	f
0 →	1 0 7 2	h
1 →	1 0 7 2	h

Total Page faults (Optimal) = 8